# Logos Experiment 10 Report

Clean Data Filtering + Wider-Shallower + Weight Tying + SwiGLU

## 1. Experiment Purpose

Experiment 10 tested whether basic quality filtering of the OpenWebText subset could improve language-modeling quality without increasing model size. The architecture was kept identical to Experiment 9: a wider-shallower decoder-only Transformer with weight tying and a SwiGLU feed-forward network.

**Main research question: Can cleaner data improve capability per training token while preserving the efficiency gains of the best lightweight architecture?**

## 2. Controlled Setup

| | |
|---|---|
| Run name | logos_exp10_clean_data_wider_shallower_weight_tying_swiglu |
| Dataset | OpenWebText clean filtered subset |
| Data filtering | Enabled |
| Tokenizer | GPT-2 tokenizer |
| Architecture | Wider-shallower decoder-only GPT-style Transformer |
| Layers | 4 |
| Embedding dimension | 288 |
| Attention heads | 9 |
| Head dimension | 32 |
| FFN type | SwiGLU |
| SwiGLU hidden size | 768 |
| Normalization | LayerNorm |
| Weight tying | True |
| Batch size | 64 |
| Block size | 256 |
| Tokens per step | 16,384 |
| Training iterations | 1500 |

## 3. Data Filtering Method

The experiment applied basic data quality filtering before tokenization. The filtering kept samples that were long enough, mostly alphabetic, not too noisy, and not highly repetitive. It also used a simple prefix-based near-duplicate guard.

| | |
|---|---|
| Minimum length | Removed very short samples with limited language-modeling signal. |
| Maximum length | Avoided extremely long or unusual samples. |
| URL count check | Reduced web-page noise and link-heavy text. |
| Alphabetic ratio | Kept text-heavy samples instead of symbol-heavy content. |

| | |
|---|---|
| Noise ratio | Removed samples with excessive non-language symbols. |
| Unique word ratio | Reduced highly repetitive text. |
| Prefix duplicate check | Removed simple near-duplicate samples. |

## 4. Final Results

| | |
|---|---|
| Parameters | 18,542,688 |
| Best iteration | 1500 |
| Best validation loss | 5.8265 |
| Final training loss | 5.7393 |
| Final validation loss | 5.8246 |
| Final validation perplexity | 338.53 |
| Checkpoint size | 70.76 MB |
| Peak VRAM | 10.54 GB |
| Training time | 129.22 sec, approximately 2.15 min |
| Training speed | approximately 192,081 tokens/sec |
| Generation speed | 115.60 tokens/sec |
| Generation latency | 8.65 ms/token |
| Average GPU power | 253.29 W |
| Estimated GPU energy | 8.98 Wh |

## 5. Training Trend

| | | | |
|---|---|---|---|
| 0 | 10.8723 | 10.8716 | 52659.36 |
| 150 | 7.1080 | 7.1014 | 1213.66 |
| 300 | 6.4731 | 6.5137 | 674.28 |
| 600 | 6.0953 | 6.1689 | 477.68 |
| 900 | 5.8710 | 5.9538 | 385.23 |
| 1200 | 5.7718 | 5.8812 | 358.24 |
| 1350 | 5.7465 | 5.8594 | 350.52 |
| 1500 | 5.7328 | 5.8265 | 339.17 |

The validation loss decreased steadily throughout the run, so the model was still learning by the end. However, the final validation quality was still weaker than Experiment 9 using the original OpenWebText subset.

## 6. Comparison Against Experiment 9

Experiment 10 used the same architecture as Experiment 9, but replaced the original OpenWebText subset with a cleaned/filtered subset. Therefore, the comparison mainly tests the effect of data filtering.

| | | | |
|---|---|---|---|
| Parameters | 18.54M | 18.54M | Tie |
| Best val loss | 5.7388 | 5.8265 | Exp 9 |
| Final val loss | 5.7490 | 5.8246 | Exp 9 |
| Final perplexity | 313.87 | 338.53 | Exp 9 |
| Checkpoint size | 70.76 MB | 70.76 MB | Tie |
| Peak VRAM | 10.54 GB | 10.54 GB | Tie |
| Training time | 132.14 sec | 129.22 sec | Exp 10 |
| Training speed | 187k tok/s | 192k tok/s | Exp 10 |
| Generation latency | 12.71 ms/token | 8.65 ms/token | Exp 10 |
| Energy | 9.06 Wh | 8.98 Wh | Exp 10 |

## 7. Scientific Interpretation

Clean data filtering did not improve validation quality in this fixed-budget run. The model trained slightly faster and generated faster, but it had worse validation loss and worse perplexity than the same architecture trained on the original subset.

**Conclusion: In this setup, the filtered data improved runtime efficiency slightly but reduced language-modeling quality. The likely cause is that filtering changed the data distribution and reduced diversity. Cleaner text is not automatically better if the filtering removes useful variety or makes the dataset less representative.**

## 8. Sample Generation

```
The future of language models. But we're more a good for example, but to be a good for our
new-one, said. It's not all we're the world. The best thing is the past a bit of the world.
He added to be a good player of a small-and-back with the team's business, but she has no
one's more likely to make the team to be able to get it out,
```

The sample shows partial grammatical structure but also repeated vague phrasing. This supports the quantitative result: the model learned basic token patterns, but the filtered data experiment did not clearly improve generation quality.

## 9. Current Leaderboard After Experiment 10

| | | |
|---|---|---|
| Best quality + storage | Experiment 5: Weight Tying + SwiGLU | Final val loss 5.7247, PPL 306.36, 19.24M params, 73.44 MB |
| Best efficiency-focused | Experiment 9: Wider-Shallower + Weight Tying + SwiGLU | 18.54M params, 70.76 MB, 187k tok/s, 9.06 Wh, val loss 5.7490 |
| Best simple architecture-shape | Experiment 8: Wider-Shallower | Faster and lower VRAM than baseline while preserving similar quality |
| Clean data test | Experiment 10 | Faster runtime, but worse loss and perplexity than Experiment 9 |

## 10. Final Notes

- Experiment 10 should not replace Experiment 9 because validation quality worsened.

- Filtering needs to be redesigned if used again. A softer filter or quality scoring approach may preserve more data diversity.

- The strongest model so far remains Experiment 5 for quality-per-storage and Experiment 9 for overall efficiency.

- Future data experiments should compare original validation data and filtered validation data separately to avoid confusing distribution effects.

### One-line conclusion

**Experiment 10 showed that basic clean data filtering made the efficient Logos model faster and slightly cheaper to run, but it hurt validation loss and perplexity, so the original data pipeline remains better for model quality under this fixed training budget.**